Ders 1 Genişletilmiş Notu: Sezgisel Arama: En İyi İlk Arama ve A* Algoritması

Ders 1 Genişletilmiş Notu: Sezgisel Arama: En İyi İlk Arama ve A* Algoritması

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Kör Aramanın Tekrarı
- 2. Kör Aramanın Sınırları: Rastgele Harita Uygulaması
- 3. Sezgisel Arama Fikri
- 4. En İyi İlk Arama (Best-First Search)
- 5. A* Araması
- 6. Izgara Üzerinde Best-First ve A*
- 7. 8 Taş Problemi İçin Sezgisel Fonksiyonlar
- 8. Esnetilmiş Problem: Siyah ve Beyaz Taşlar
- 9. Şehir Grafı Üzerinde Arama Algoritmaları (S'den G'ye)
- 10. Labirent Çalışma Sorusu
- 11. A*'ın Optimalliği
- 12. Uygulama: Algoritmaların Karşılaştırılması ve Robot Keşfi
- Örnek Sorular

1. Kör Aramanın Tekrarı

Hoca derse, geçen hafta arama problemlerinin kör arama algoritmalarıyla nasıl çözüldüğünü hatırlatarak başladı. Bu hafta, yapay zekanın artık işin içine girdiği sezgisel arama (heuristic search) konusuna geçilecekti.

Bir arama problemi için durum uzayı ağacı (state space tree) kurulur. Başlangıç durumundan, dallanma sayısına (branching factor) göre farklı ihtimaller çıkar. Bunlardan herhangi biri açıldığında yeni ihtimaller doğar ve ağaç sürekli genişler. Sonuçta çok büyük bir arama uzayı ortaya çıkar.

Hoca temel tanımları tekrar etti:

- **Durum uzayı:** Olabilecek bütün ihtimalleri gösteren yapı.
- **Operatörler:** Bir durumdan başka bir duruma geçişi sağlayan hamleler.
- **Başlangıç ve hedef durumu.**
- **Maliyet:** Operatörleri uygulamanın, yani başlangıçtan hedefe doğru yapılan aramanın maliyeti.
- **Çözüm:** Başlangıçtan hedefe giden yol. Maliyet ile çözüm farklı şeylerdir.
- **Optimum yol:** En düşük maliyetli çözüm yolu.

Arama stratejileri dört boyutta değerlendirilmişti: Tam (complete) mı, yani bir çözüm varsa buluyor mu? Zaman karmaşıklığı nedir? Bellek (space) karmaşıklığı nedir? Optimum mu, yani ilk bulunan çözüm en düşük maliyetli çözüm mü? Zaman ve bellek karmaşıklığında üç büyüklüğe bakılır:

- b: bir düğümden çıkabilecek en fazla dal sayısı (branching factor).

- d: en az maliyetli çözümün derinliği.
- m: arama uzayının en büyük derinliği. Bu derinlik bazen sonsuz olabilir.

1.1 Neden "Kör" Diyoruz?

Bu algoritmalara kör, mekanik, bilgisiz (blind, uninformed) denmesinin nedenini hoca şöyle açıkladı. Genişlik öncelikli aramada (breadth-first search, BFS) genelde bir kuyruk yapısı kullanılır. Bulunduğumuz düğümden gidilebilecek komşulara geçmenin maliyetleri eşit olsun. Algoritma bir bilgiye göre değil, kendisine verilen direktife göre hareket eder. Çözüm belki çok yakındadır, ama algoritma önce o seviyedeki bütün düğümleri bitirmek zorundadır. Bu yüzden çözüme çok yaklaşmışken bir anda ondan uzaklaşabilirsiniz.

Derinlik öncelikli arama (depth-first search, DFS) da benzer şekilde körü körüne bir dal boyunca en dibe kadar iner. Belki çözüm hemen yan taraftadır, çözüme çok yaklaşmışsınızdır, ama algoritma başka bir dala dağılıp gider.

2. Kör Aramanın Sınırları: Rastgele Harita Uygulaması

Hoca, geçen derste de bakılan, rastgele oluşturulmuş bir harita örneğini yakından inceledi. Önceki ağaç örneklerinde yan düğüme geçmek ile alt düğüme geçmek eşit maliyetliydi. Bu haritada ise kenarlarda kilometreler, yani farklı maliyetler var. Örnekte 12 numaralı düğümden 1 numaralı düğüme gidiliyor.

- **BFS:** 12'den 4'e, 4'ten 2'ye, 2'den 5'e, sonra 1'e gidiyor. Yol 128 maliyetli ve 37 adımda bulunuyor. 37 adım şöyle oluşuyor: Algoritma önce 12'den doğrudan gidilebilen 4, 11, 9 gibi düğümlere tek tek bakıyor, sonra 4'ten gidilebilenleri geziyor. Böylece başlangıcın önce en yakınlarını, sonra biraz daha uzaklarını katarak, ağaç gibi genişleyerek 1'e ulaşıyor. İlk bakışta 128 maliyet fena değil gibi görünüyor. Ama BFS yalnız her adımın maliyeti eşitse optimal çözümü bulur. Burada maliyetler eşit olmadığı için 5 atlamalı bir yol buldu, ama bu yol en az maliyetli yol değil.
- **Düşük maliyetli arama (uniform cost search, UCS):** Normal kuyruk yerine öncelikli kuyruk (priority queue) kullanıyor. Maliyetler pozitif olduğu için optimum çözümü, yani 110 maliyetli yolu buluyor. Ama çok daha fazla adım atıyor. Arama uzayı büyüdükçe ve derinleştikçe bu adım sayısı çok artar. Bir noktadan sonra UCS yönetilemez bir algoritma hâline gelir. Hoca, UCS'nin yapı olarak Dijkstra algoritmasına büyük oranda benzediğini de belirtti.
- **Best-first ve A*:** İkisi de aynı optimum sonuca çok daha kısa sürede ulaşıyor.
- **DFS:** 12, 11, 9, 15 diye ilerliyor. Seçimler biraz tesadüfi; hocaya göre muhtemelen düğüm numarası küçük olan tercih ediliyor. 15'e geldiğinde 1 çok uzakta değil, neredeyse iki adım ötede. Ama algoritma bilgisiz olduğu için "çözümün kapısına kadar" gidip geri dönüyor, 18'e, 17'ye, 13'e uzaklaşıyor.

Hoca bu örneği, kör arama algoritmalarının sizi sonuca ulaştırabileceğini, ama optimum sonuçtan uzaklaşabileceğini gösteren güzel bir durum olarak niteledi. Bilgi olmadığı için "kafası kesik tavuk gibi" bir arama yapılır.

Hoca ardından MATLAB'da yazılmış bu karşılaştırma kodunu Python'a çevirdiğini söyledi ve canlı denedi. 20 düğümlü, 35 yollu rastgele bir graf oluşturdu. 5'ten 16'ya gidilen denemede A* ve best-first 3 adımda sonuca ulaştı. Kör algoritmalar 11 ve 17 adım gibi daha fazla adım attı. 16 yerine 13'e gidilen ikinci denemede de sonuç benzerdi: Biri 29, diğeri 11 adım atan kör algoritmalara karşı sezgisel algoritmalar hem adım sayısı hem en kısa rota açısından öne çıktı. Grafikte yeşil renk UCS'yi, kırmızı BFS'yi gösteriyordu.

Hocanın çıkardığı sonuç şudur: Elinizde değişmeyen, statik ve küçük bir graf varsa kör algoritmalardan bazılarını kullanabilirsiniz. Ama gerçek dünyada, özellikle dinamizmin olduğu ve arama uzayının sürekli genişlediği problemlerde, sizi daha hızlı sonuca götürecek yöntemlere ihtiyaç vardır. Bu yöntemlerde bir düğümden sonrakine geçerken, körü körüne veya belli bir talimatı izleyerek düğüm seçilmez. Örneğin BFS üç komşuyu sırayla gezer. Oysa çözüm öyle bir yerdedir ki üçünü de gezmeye gerek yoktur. Sezgisel algoritmalar, sezgisel bir bilgiyle bir sonraki düğümü seçmenizi sağlar.

3. Sezgisel Arama Fikri

Sezgisel aramada problemin yapısı aynıdır: yine durum uzayı ağacı, başlangıç durumu ve operatörler vardır. Fark şudur: Bir operatörün çalışması, yani bir durumdan diğerine geçiş, körü körüne bir kuralla değil sezgisel bir kuralla belirlenir.

Hoca slayttaki bir alıntıyı okudu: "Özel buluşlara çok değer vermiyorum. En çok arzu ettiğim şey icat etme sanatını mükemmelleştirmek ve problemin çözümünü bulmaktan ziyade çözüm yöntemlerini bulmaktır. Çünkü tek bir yöntem sayısız çözümü kapsar." Buradaki vurgu, farklı çözüm yöntemlerinin ön plana çıkmasıdır.

Feldman ve Feigenbaum'a göre sezgisellik şudur: Durum uzayı çok büyük olduğunda veya çok dinamik bir şekilde genişlediğinde, çözüm aramasını sınırlayan her türlü kural, strateji, hile, sadeleştirme ve diğer etmenlerin kullanımı.

Hoca bunların hepsini tek cümlede özetledi: Sezgi, "hedefe daha yakın bir şekilde nasıl gidebilirim?" sorusunun tahminidir. Bir noktadan diğerine geçerken öyle bir düğüm seçmeliyim ki beni nihai hedefime daha hızlı götürsün.

Bu derste iki sezgisel algoritma işlendi: en iyi ilk arama (best-first search) ve A. *Best-first search yerel minimumlara (local minima) takılabilir. A* ise bu yerel minimumlardan kurtulabilme özelliğine sahiptir. Yerel arama (local search) algoritmaları ise başlı başına ayrı bir konu olup gelecek hafta işlenecek.

4. En İyi İlk Arama (Best-First Search)

4.1 Algoritma

Best-first search, BFS ve DFS'dekine benzer bir yapı (kuyruk veya yığın) kullanır. Adımları şöyledir:

1. Kök düğümü kuyruğa koy.
2. Hedef bulunana kadar veya kuyruk boş olmadığı sürece devam et.
3. Kuyruktan ilk durumu çek. Çektiğin durum hedefse "bulundu" de ve çık.
4. Çekilen durumdan gidilebilecek bütün durumları kuyruğun sonuna ekle. (Başına eklenseydi DFS olurdu.)
5. Kuyruktaki durumlara bir değerlendirme fonksiyonuna göre puan ver ve durumları puanlarına göre küçükten büyüğe sırala.

Hoca son maddenin, öbür algoritmalarda olmayan yenilik olduğunu vurguladı. Bulunduğum düğümden üç yere gidebileceğimi düşünelim. Her birinin puanını alırım: Şuraya gidersem 8, buraya gidersem 5, buraya gidersem 18. Bu puanları küçükten büyüğe sıralayıp karar veririm ve yeni duruma geçerim.

Bu, UCS'deki maliyete biraz benzer. Ama UCS'deki maliyet, şimdiye kadar harcanan efordur: başlangıçtan bulunduğum noktaya kadar sarf ettiğim maliyet. Gelecekteki maliyeti dikkate almaz. Best-first search'te değerlendirme fonksiyonu ise sezgisel (heuristic) bir $h(n)$ fonksiyonunun sonucudur:

$$f(n) = h(n)$$

$h(n)$, mevcut durumun hedefe olan tahmini uzaklığıdır. Hoca bu tahminin **iyimser** olması gerektiğini vurguladı. Harita problemlerinde tipik örnek, hedefe olan kuş uçuşu mesafedir. A noktasından L noktasına gideceğinizi, yolda B, E, K gibi noktalara uğrayabileceğinizi düşünün. Her adımda, $f(n) = h(n)$ fonksiyonundan bilgi alarak, hedefe en yakın görünen durumu seçersiniz.

4.2 Romanya Örneği

Hoca kitabın klasik örneğini kullandı: Romanya'daki Arad şehrinden Bükreş'e (Bucharest) gitmek. Haritadaki kenar değerleri gerçek yol mesafeleridir (karayolu, demiryolu gibi). Bunlardan ayrı olarak ekstra bir bilgi verilir: her şehirden Bükreş'e olan kuş uçuşu mesafe (straight-line distance). Best-first search yalnız bu ikinci bilgiye, yani $h(n)$'e bakarak karar verir.

Arama adım adım şöyle ilerler (Python ile doğrulandı):

Adım	Kuyruktan çekilen	Çekildikten sonra kuyruk (h değerleriyle)
1	Arad (366)	Sibiu 253, Timisoara 329, Zerind 374
2	Sibiu (253)	Fagaras 176, Rimnicu Vilcea 193, Timisoara 329, Zerind 374, Oradea 380, (Arad 366)
3	Fagaras (176)	Bükreş 0, Rimnicu Vilcea 193, ... , (Sibiu 253)
4	Bükreş (0)	Hedef, arama durur.

- Arad'dan başlanır. Arad'ın Bükreş'e kuş uçuşu uzaklığı 366'dır. Arad'dan gidilebilecek üç yer vardır: Sibiu (253), Timisoara (329), Zerind (374). Fiziki olarak yapabileceğiniz hamleler bunlardır. Hoca burada karşılaştırma yaptı: BFS olsaydı belki alfabetik sıraya göre önce S'yi, sonra T'yi, sonra Z'yi gezerdi. Best-first search ise h değeri en küçük olan Sibiu'yu seçer.
- Sibiu kuyruktan çıkınca ondan gidilebilecek şehirler kuyruğa girer: tekrar Arad, Fagaras, Oradea, Rimnicu Vilcea. Kuyrukta Timisoara ve Zerind de olduğu için toplam 6 eleman vardır. Bunların en küçüğü Fagaras (176) olduğu için Fagaras çekilir.
- Fagaras'tan Sibiu ve Bükreş kuyruğa girer. Bükreş'in kuş uçuşu uzaklığı zaten 0'dır. En küçük değer o olduğu için Bükreş seçilir ve hedefe ulaşıldığı için algoritma durur.

Bulunan yol Arad → Sibiu → Fagaras → Bükreş'tir. Maliyeti:

$$140 + 99 + 211 = 450$$

Peki bu optimum bir çözüm mü? Hoca'nın cevabı: Hayır. Algoritma bizi çok zahmete sokmadan bir çözüm buldu, toplam dört adımda (Arad, Sibiu, Fagaras, Bükreş çekildi) hedefe gitti. Yolda önümüze çıkan diğer seçenekleri hiç açmadı. BFS'ye göre çok daha az hamle yaptı. Ama yalnız kuş uçuşu mesafeye baktığı için daha kısa bir yolu kaçırdı. Gerçekte şu yol vardır:

$$\text{Arad} \to \text{Sibiu} \to \text{Rimnicu Vilcea} \to \text{Pitesti} \to \text{Bükreş}: \quad 140 + 80 + 97 + 101 = 418$$

Yani algoritma yerel minimuma takılmıştır.

Hoca bu noktada bir ad karışıklığına da dikkat çekti: Best-first search'ü BFS diye kısaltmak genişlik öncelikli aramayla karışıyor. Bu yüzden best-first search için "BeFS" denebilir. Bu notta BFS yalnız genişlik öncelikli arama anlamında kullanılıyor.

4.3 Best-First Search'ün Özellikleri

- **Tam değildir.** Döngülere takılabilir. Örneğin Arad → Sibiu → Fagaras → Sibiu veya Arad → Sibiu → Arad gibi bir döngü, maliyetler denk gelirse oluşabilir. Hedefe aynı uzaklıkta iki durumun bulunduğu bir uzay düşünün: Algoritma bu ikisi arasında gidip gelerek döngüye girer.
- **Zaman karmaşıklığı:** Yine bir ağaç yapısı vardır ve en kötü durumda $O(b^m)$ olur. Ama iyi bir sezgisel kural verirseniz zaman çok iyileşir. Harita problemlerinde iyi bir sezgisel kural, iki konum arasındaki en kısa mesafe olan kuş uçuşu (Öklid) uzaklığıdır. Zaman açısından BFS ve DFS'ye göre ciddi biçimde daha hızlı sonuca götürür.
- **Bellek karmaşıklığı:** Bütün durumları hafızada tutmak zorunda olduğu için $O(b^m)$'dir. Hocanın deyişiyle "bundan kaçış yok".
- **Optimal değildir.** Yalnız sezgiye bakarak ilerlemek sizi gerçek çözümden uzaklaştırabilir ve yerel minimuma takılırsınız. Daha iyi bir çözüm varken ondan kaçarsınız.

  Ek bilgi (derste anlatılmadı): Ders sonunda hocanın kitaptan gösterdiği açgözlü arama (greedy search), bu derste best-first search diye anlatılan $f(n) = h(n)$ algoritmasıdır. Russell ve Norvig'in kitabında "best-first search" genel bir şablonun

adıdır; hangi düğümün açılacağını bir $f(n)$ fonksiyonu belirler. $f(n) = h(n)$ seçilince açgözlü best-first arama, $f(n) = g(n) + h(n)$ seçilince A* elde edilir.

5. A* Araması

5.1 Fikir: Harcanan ve Kalan Maliyeti Birlikte Kullanmak

A* algoritması, best-first search'teki değerlendirme fonksiyonunu genişletir. Hocaya göre A*, UCS ile best-first search'ün bir birleşimi gibi düşünülebilir: "Mevcut harcadığım maliyeti de dikkate alayım, sezgisel maliyeti de dikkate alayım, ikisinin birleşiminden karar vereyim."

$$f(n) = g(n) + h(n)$$

- $g(n)$: başlangıçtan n düğümüne kadar harcanan gerçek maliyet.
- $h(n)$: n'den hedefe tahmini (sezgisel) maliyet.

Algoritmanın adımları best-first search ile aynıdır. Yalnız sıralama $f(n) = g(n)+h(n)$ değerine göre yapılır.

5.2 Romanya Örneği

Arad'dan yine başlanır. Henüz harcanan maliyet yoktur, sezgisel maliyet 366'dır: $f = 0 + 366 = 366$. Arad'dan Sibiu'ya gerçek maliyet 140, Timisoara'ya 118, Zerind'e 75'tir. Kuş uçuşu uzaklıklar sırasıyla 253, 329 ve 374'tür. Toplamlara bakılır ve en küçük olan seçilir.

A* izlemesi (Python ile doğrulandı):

Adım	Kuyruktan çekilen	$f = g + h$	Çekildikten sonra kuyruğa girenler
1	Arad	$0 + 366 = 366$	Sibiu $140 + 253 = 393$, Timisoara $118 + 329 = 447$, Zerind $75 + 374 = 449$
2	Sibiu	$140 + 253 = 393$	Rimnicu Vilcea $220 + 193 = 413$, Fagaras $239 + 176 = 415$, Arad $280 + 366 = 646$, Oradea $291 + 380 = 671$
3	Rimnicu Vilcea	$220 + 193 = 413$	Pitesti $317 + 100 = 417$, Craiova $366 + 160 = 526$, Sibiu $300 + 253 = 553$
4	Fagaras	$239 + 176 = 415$	Bükreş $450 + 0 = 450$, Sibiu $338 + 253 = 591$
5	Pitesti	$317 + 100 = 417$	Bükreş $418 + 0 = 418$, Rimnicu Vilcea $414 + 193 = 607$, Craiova $455 + 160 = 615$
6	Bükreş	$418 + 0 = 418$	Hedef, arama durur.

Hoca adımları şöyle anlattı. Sibiu çekildikten sonra kuyruğa tekrar Arad, Fagaras, Oradea ve Rimnicu Vilcea gelir; Timisoara ve Zerind de kuyrukta durmaktadır. Hepsinin harcanan ve sezgisel maliyetleri doldurulur. En küçük f değeri 413 olduğu için Rimnicu Vilcea seçilir. Rimnicu Vilcea açılınca yeni ihtimaller çıkar. Bu kez en küçük değer bir adım gerideki Fagaras'tadır (415), ona gidilir. Fagaras'tan Bükreş'e doğrudan bir yol görülür ve sezgisel maliyeti 0 olan bir çözüm bulunur. Ama bunun toplamı 450'dir. Algoritma "daha kısa bir yol var" der ve Pitesti'ye (417) gider. Pitesti'den sonra Bükreş yine görünür, bu kez $f = 418$ ile. Kuyruktaki en küçük değer 418 olduğu için o seçilir ve hedefe ulaşılır.

Bulunan yol Arad → Sibiu → Rimnicu Vilcea → Pitesti → Bükreş, maliyeti 418'dir ve optimumdur. Hocanın saymasıyla arama beş adımda (beş düğüm açılarak) çözümü buldu. Çözümün kendisi dört atlamalıdır.

> **Ders sonrası not:** Hoca A* anlatımının ilk adımında "sezgisel maliyete baktığımızda 253 olduğu için Sibiu'yu seçtik" dedi. A*'da seçim yalnız h'ye göre değil $f = g+h$'ye göre yapılır: Sibiu 393, Timisoara 447, Zerind 449 olduğu için Sibiu seçilir. Sonuç aynıdır ama gerekçe f değeridir. Ayrıca hoca bir yerde "toplam $f(n)$ hangisinde fazlaysa onu tercih ediyoruz" dedi; doğrusu en küçük $f(n)$ seçilir, nitekim hemen ardından "413 en küçük olduğu için" diye devam etti.
>
> **Ders sonrası not:** Best-first anlatımında Oradea'nın kuş uçuşu uzaklığı altyazıda "385" diye geçiyor. Kitaptaki tabloda bu değer büyük olasılıkla 380'dir; yukarıdaki tablolar 380 ile hesaplandı. Sonuç değişmez.

5.3 A*'ın Özellikleri

- **Tamdır.** Hocaya göre A* döngüye takılmaz, çünkü karar $f(n) = g(n)+h(n)$'e göre verilir. Bir döngüye girildiğinde, h değeri küçük olsa bile o ana kadarki toplam maliyet işin içine girdiği için döngülü yolların f değeri hep diğerlerinden büyük olur ve tercih edilmez. Örnek: Arad'ın kuş uçuşu uzaklığı 366'dır. Arad → Sibiu → Arad yolunda harcanan maliyet $140 + 140 = 280$ olduğu için $f = 280 + 366 = 646$'ya çıkar. Bu yolu seçme ihtimali neredeyse sıfıra düşer. Doğru yol bizi hedefe yaklaştırırken, döngü bizi uzaklaştırmaya başlar.
- **Zaman ve bellek:** Yine $O(b^m)$'dir, ama iyi sezgisel kurallarla zaman çok hızlanır.
- **Optimaldir, ama bir şartla.** Düzgün, daha doğrusu **iyimser** bir değerlendirme fonksiyonu kullanılmalıdır. Böyle bir fonksiyon varsa A* sizi optimal sonuca götürür.

  Hocanın vurgusu: Her A* illa optimal sonuca götürecek diye bir şey yok. $h(n)$ fonksiyonunu yanlış tasarlarsanız optimal sonuca götürmeyebilir. Romanya örneğinde kuş uçuşu mesafe kullanıldığı için sorun yok.

6. Izgara Üzerinde Best-First ve A*

Hoca ardından bir ızgara (grid) örneği gösterdi. Amaç S karesinden G karesine gitmek. Arada engeller var. BFS, DFS veya UCS de uygulanabilir, ama burada best-first ve A* ile çözülecek.

İlk iş, bütün karelerin hedefe olan Manhattan uzaklığını yazmaktır: yatay ve düşeydeki kare sayıları toplanır, aradaki engeller dikkate alınmaz. Hoca kareleri tek tek saydı: biri 7, biri 6, sonra 5, 4, 3 diye azalıyor. Bu da kuş uçuşu mesafeye benzer bir fikirdir, ama karekökle uğraşmamak için Öklid uzaklığı yerine Manhattan uzaklığı kullanılmıştır.

6.1 Best-First ile

Best-first search'te $f(n) = h(n)$'dir, $g(n)$ yoktur. S kuyruktan çıkınca gidilebilecek üç kare vardır: ikisinin değeri 6, birinin 8. Hoca iki 6'yı ayırt etmek için "6-1" ve "6-2" diye adlandırdı. Burada aynı değerli iki durum var. Hoca bunun, hedefe aynı uzaklıktaki iki durum arasında sonsuz döngüye girme riskini hatırlattığını söyledi.

6-1 seçilsin. 6-1'den bir 7 gelir. Kuyrukta 6-2, 7 ve 8 vardır; en küçük 6-2 seçilir. 6-2'den bir 7 daha gelir (7-1, 7-2). 7-1 seçilir, ondan 8'ler gelir. Sonra 7-2 seçilmek zorunda kalınır. 7-2 üçüncü bir 6 getirir. 6 seçilir, sonra 5 gelir, sonra 4, 3, 2 ve ardından değerler yeniden 3, 4, 5, 6, 5, 4 diye değişir. Engelin arkasında dolaşılır.

Normalde kısa bir yoldan çözüme ulaşılabilirdi. Ama best-first search yalnız $h(n)$'e bağlı $f(n)$'e baktığı ve önüne hep 8'den küçük değerler geldiği için kendine böyle dolambaçlı bir rota bulur. Bu yol başlangıçtan hedefe iyi bir yol değildir, optimum değildir. Bu örnekte bir döngüye takılmadan çözüm bulundu. Hocanın vurguladığı nokta: Önceki haritada olduğu gibi burada da algoritma önce çözüme yaklaşır gibi oldu, ama o üçüncü 6'yı bulmak bütün olayın seyrini değiştirdi ve optimal çözüm bulunamadı.

6.2 A* ile

A'*da* $f(n) = g(n) + h(n)$*'dir. S'den bir birim maliyetle iki komşu kareye gidilebilir:* $1 + 6 = 7$*. Biri seçilir. Ardından* $2 + 7$*,* $3 + 8$*,* $5 + 4$ *gibi değerler hesaplanır. Bir süre değerler iyileşir gibi görünür (*$3 + 5 = 8$*,* $3 + 4 = 7$ *gibi), ama engelin arkasına gelindiğinde bir anda* $4 + 9 = 13$*'e çıkılır. Bu noktada kuyrukta daha az maliyetli bir seçenek (11) vardır. A* oradan devam eder ve sonunda daha kısa olan optimal yolu bulur.

İşaretlenen bütün kutucuklar sayılırsa deneme sayısı yine de az değildir. Ama aynı problemi BFS veya UCS ile çözmeye kalksak çok daha fazla kare açmamız gerekirdi.

7. 8 Taş Problemi İçin Sezgisel Fonksiyonlar

7.1 İki Sezgisel Fonksiyon

Hoca daha önce görülen 8 taş (8-puzzle) problemine döndü. Durum geçişleri boşluğu hareket ettirerek yapılır. Bu problem için iki sezgisel kural önerilir:

- $h_1(n)$: yerinde olmayan taşların sayısı.
- $h_2(n)$: taşların hedef konumlarına olan uzaklıklarının (Manhattan uzaklığı) toplamı.

Slayttaki başlangıç durumu şudur:

```
7 2 4
5 _ 6
8 3 1
```

h_1 **hesabı:** Hoca taşlara tek tek baktı: 7 yerinde değil, 2 yerinde değil, 4 yerinde değil, 5 de, 6 da... 8 taşın hiçbiri yerinde değil. Yani $h_1 = 8$. Yerinde olan bir taş olsaydı h_1 7 olurdu.

h_2 **hesabı:** 7'nin olması gereken yere uzaklığı 3, 2'nin 1, 4'ün 2'dir. Hoca ilk üç taşı saydı ve toplamın 18 olduğunu söyledi. Bütün taşlar için hesap (ara adım eklendi, Python ile doğrulandı):

Taş	7	2	4	5	6	8	3	1
Uzaklık	3	1	2	2	3	2	2	3

$$h_2 = 3 + 1 + 2 + 2 + 3 + 2 + 2 + 3 = 18$$

Arama uzayında her hamle (boşluğu yukarı, aşağı, sağa veya sola kaydırmak) 1 birim maliyetlidir; bu $g(n)$'dir. Bu iki sezgisel fonksiyondan gelen bilgiyle A* araması yapılabilir.

Hoca "düğüm" ile neyin kastedildiğini de gösterdi. Bulunduğumuz durumdan, örneğin boşluğu yukarı kaydırırsak 2 aşağı iner ve yeni bir durum oluşur. Başka hamlelerle başka durumlar oluşur. Her durumun alt durumları, onların da alt durumları vardır. Böylece 12 veya 24 derinlikli ağaçlar ortaya çıkar.

Slaytta ayrıca hedefe çok yakın örnek durumlar için h_1 ve h_2'nin 1 çıktığı örnekler de vardı.

7.2 Baskınlık ve Deneysel Sonuçlar

Birden fazla sezgisel kural varsa şöyle karar verilir: Bütün durumlar için $h_2(n) \geq h_1(n)$ ise h_2, h_1'e baskındır (dominant). Bu durumda h_2'yi kullanmak daha uygundur.

Kitaptaki deneysel sonuçlar (açılan düğüm sayısı):

Çözüm derinliği	IDS	A*(h_1)	A*(h_2)
12	yaklaşık 3,6 milyon	227	73
24	hesaplanamayacak kadar çok	39.135	1.641

Yani derinlik 12 iken yinelemeli derinleşen arama (iterative deepening search, IDS) milyonlarca düğüm açarken A*, h_1 ile 227, h_2 ile yalnız 73 düğümde sonuca ulaşır. Derinlik 24'e çıkınca fark daha da büyür.

> **Ders sonrası not:** Hoca h_2'nin tercih edilme nedenini "size daha optimistik bir çözüm bilgisi veriyor" diye açıkladı. Doğrusu tersidir: Baskın fonksiyon her durumda daha büyük, yani gerçek kalan maliyete daha yakın bir tahmin verir; bu anlamda daha az iyimserdir. İki fonksiyon da kabul edilebilir (gerçek maliyeti aşmayan) olduğu sürece, gerçeğe daha yakın tahmin A*'a daha az düğüm açtırır. Tablodaki 73'e karşı 227 farkı bundandır.

8. Esnetilmiş Problem: Siyah ve Beyaz Taşlar

Hoca esnetilmiş problemler (relaxed problems) başlığı altında bir problem daha gösterdi. Bir dizide siyah (B) ve beyaz (W) taşlar ve bir boşluk vardır. Amaç B'lerle W'lerin yerini değiştirmektir. Kısıtlar şunlardır: Bir taş yanındaki boşluğa kayabilir veya yanındaki bir taşın üzerinden atlayarak boşluğa geçebilir. İstediğiniz taşı istediğiniz yere taşıyamazsınız.

Bu problem için bir $h(n)$ önermek çalışma sorusu olarak verildi. Hoca, "hocam ben $h_1(n) + h_2(n)$ gibi bir şey düşündüm" şeklinde öneriler getirilebileceğini söyledi.

9. Şehir Grafı Üzerinde Arama Algoritmaları (S'den G'ye)

Aradan sonra hoca harita tarzında yeni bir problem gösterdi. Kenarlardaki sayılar gerçek mesafeler, düğümlerin yanındakiler ise G'ye kuş uçuşu mesafelerdir. Amaç S'den G'ye gitmektir.

Kenar maliyetleri (Şekil-1): S–A 3, S–D 4, A–B 4, A–D 5, B–C 4, B–E 5, D–E 2, E–F 4, F–G 3.

G'ye kuş uçuşu uzaklıklar: S 11, A 10,4, B 6,7, C 4, D 8,9, E 6,9, F 3, G 0.

9.1 BFS

BFS'de S'den A'ya ve D'ye gidilir. A'ya D üzerinden de gidilebilir, D'den tekrar A'ya veya E'ye gelinebilir. Ağaçta düğümler adım sırasıyla numaralanır. G'ye 21., 22. ve 24. adımlarda ulaşılabildiği görülür; bunlar farklı çözümlerdir. Hoca bu numaraların mesafe değil, hangi düğüme hangi sırayla ulaşıldığını gösteren indisler olduğunu vurguladı. Bir yerden başlayıp aynı harfe tekrar gelinebildiği için bu ayırt etme gerekir.

Hangisi optimum? BFS bunu bulamayabilir, çünkü maliyetler eşit değildir. Hoca iki yolu karşılaştırdı:

- S → D → E → F → G: $4 + 2 + 4 + 3 = 13$.
- S → A → D → E → F → G: $3 + 5 + 2 + 4 + 3 = 17$ (ara adım eklendi).

Dört adımda gidilebilen bir yol daha ucuz olabildiği gibi, G ağacın daha sığ bir yerinde de olabilirdi; ama oradaki kenarlar çok uzunsa o da optimum olmazdı.

BFS'de kuyruğun başından çıkarılır, sonuna eklenir. S çıkar, A1 ve D'ler eklenir. A1 çıkar, B3, D4 gibi düğümler eklenir. Ağaç öyle derinleşir ki bir seviyeye gelindiğinde neredeyse aşağıdaki bütün düğümleri ekleyecek kadar üstel bir artış olur. Örnekte G21'den önce G24'e ulaşılıp o kuyruktan çıkarılsaydı, "çözüm" denip durulurdu ama bu optimum çözüm olmazdı. Bu, maliyetler eşit olmadığında BFS'nin optimum çözüme götürmediği bir örnektir.

9.2 DFS

DFS'de S'den başlanır, A ve D eklenir. DFS belleği çok daha iyi kullanır, doğrusal bir bellek yaklaşımı vardır. Ama bu örnekte G22'yi bulur; oysa G21 daha iyi bir çözümdü.

9.3 UCS

UCS'de öncelikli kuyruk vardır ve G, başlangıçtan uzaklığı 13 olan yoldan bulunur. Hoca, BFS'de 21. adımda ulaşılan şeye burada 13 maliyetle ulaşıldığını söyledi. Ama UCS'de de bellek kullanımı oldukça fazladır.

9.4 A*

A*'da her düğüm için harcanan fiziki maliyet ve kuş uçuşu mesafe toplanır. İzleme (Python ile doğrulandı):

Adım	Kuyruktan çekilen yol	$f = g + h$	Çekildikten sonra kuyruk
1	S	$0 + 11 = 11$	S-D $4 + 8{,}9 = 12{,}9$; S-A $3 + 10{,}4 = 13{,}4$
2	S-D	$12{,}9$	S-D-E $6 + 6{,}9 = 12{,}9$; S-A $13{,}4$; S-D-S $8 + 11 = 19$; S-D-A $9 + 10{,}4 = 19{,}4$
3	S-D-E	$12{,}9$	S-D-E-F $10 + 3 = 13$; S-A $13{,}4$; S-D-E-D $16{,}9$; S-D-E-B $11 + 6{,}7 = 17{,}7$; ...
4	S-D-E-F	13	S-D-E-F-G $13 + 0 = 13$; S-A $13{,}4$; ...
5	S-D-E-F-G	13	Hedef.

Hoca adımları şöyle anlattı: S kuyruktan çıkınca D'ye ve A'ya gidilebilir. D'ye gitmenin maliyeti 4, D'den G'ye tahmini kuş uçuşu mesafe 8,9; toplam 12,9. A'ya gitmenin maliyeti 3, A'dan G'ye tahmin 10,4; toplam 13,4. Kuyruktan D (12,9) çıkar. D'den sonra E ve A gelir. A daha önce 13,4 olarak gelmişti, şimdi 19,4 olarak tekrar gelir. E ise 12,9 olarak gelir ve seçilir. E'den sonra B ve F gelir, F alınır. Ondan sonra en küçük olan G (13,0) alınır. h değeri 0 olan G'ye ulaşıldığında aranan noktaya ulaşılmış olur ve A* optimum yolu verir: S–D–E–F–G, maliyet 13.

Hoca, A*'da da hafıza kullanımının genişleyerek gittiğini, ama adım sayısının çok derinleşmediğini, çok artmadığını belirtti.

9.5 Sınavda Beklenen Çözüm Biçimi

> **Hocanın vurgusu:** Bu tarz bir problemi sınavda çözmeniz istenirse, her adımda kuyruğun mevcut durumunu ve kuyruktan seçilen elemanı yazmanız beklenir. A* için gelişigüzel bir çözüm kabul edilmez.

Hoca slayttaki örnek çözüm tablosunu anlattı. Başlangıçta kuyrukta yalnız S vardır: harcanan maliyet 0, kuş uçuşu maliyet 11. S kuyruktan çıkacağı için kalın (bold) işaretlenir. Hangi yol kuyruktan çıkacaksa o böyle işaretlenir. Sonra S'den gidilebilecek A ve D yazılır: S'den D'ye 4 maliyet ve 8,9 sezgisel maliyet; S'den A'ya 3 maliyet ve 10,4 sezgisel maliyet. Hangisi çıkıyorsa o seçilir ve ondan gidilebilecek yollar eklenir. Örneğin S-D-S yolu da bir ihtimaldir: $4 + 4 = 8$ harcanır, S'nin kuş uçuşu değeri 11'dir, $f = 19$ olur. S-D-A ve S-D-E de birer ihtimaldir. Her satırda en küçük f seçilir.

Hoca, döngülü yolların kendiliğinden elendiğini burada da gösterdi: S-D-S gibi yollarda maliyet hep artar. 19 gibi değerler bulunur, ondan küçük değerler hep geldiği için bu yollar bir yerden sonra geride kalır. Kuyruk her adımda genişler ve hedefe ulaşıldığında durulur. Slayttaki tablonun sonucu: 15 birim maliyetle S, A, B, E, F, G yolu.

> **Ders sonrası not:** Sınav tablosu slaytındaki graf, Şekil-1'den farklı kenar maliyetleri kullanıyor (kısa nottaki slayt bilgisine göre B–E 1, D–E 5). Bu grafta optimal yol S–A–B–E–F–G, maliyet $3 + 4 + 1 + 4 + 3 = 15$ olur ve hocanın söylediği sonuç budur.

Şekil-1'in maliyetleriyle aynı yolun maliyeti $3 + 4 + 5 + 4 + 3 = 19$ olur, optimal yol ise S–D–E–F–G (13) olur. Ayrıca hoca tablo anlatırken A'nın sezgisel değerini bir yerde "10,9" diye okudu; tablodaki değer 10,4'tür.

Sınav grafı için A* izlemesi (Python ile doğrulandı; kalın satırlar kuyruktan çekilen yol):

Adım	Çekilen yol	f	Kuyrukta kalanlar (bir kısmı)
1	**S**	11	S-D 12,9; S-A 13,4
2	**S-D**	12,9	S-A 13,4; S-D-E $9 + 6{,}9 = 15{,}9$; S-D-S 19; S-D-A 19,4
3	**S-A**	13,4	S-A-B $7 + 6{,}7 = 13{,}7$; S-D-E 15,9; S-A-D 16,9; S-A-S 17; …
4	**S-A-B**	13,7	S-A-B-E $8 + 6{,}9 = 14{,}9$; S-A-B-C $11 + 4 = 15$; S-D-E 15,9; …
5	**S-A-B-E**	14,9	S-A-B-C 15; S-A-B-E-F $12 + 3 = 15$; S-A-B-E-B 15,7; …
6	**S-A-B-C**	15	S-A-B-E-F 15; …
7	**S-A-B-E-F**	15	S-A-B-E-F-G $15 + 0 = 15$; …
8	**S-A-B-E-F-G**	15	Hedef.

Bu sıra hocanın gezinti anlatımıyla uyumludur: Önce S'den D'ye gidildi; A kuyruğa eklenmişti ama henüz gidilmemişti. Sonra S'den A'ya, A'dan B'ye, B'den E'ye gidildi; sonra C denendi, ardından F ve G'ye ulaşıldı. Adım 5'ten sonra C ile F eşit f değerine (15) sahiptir; eşitlik hangi sırayla bozulursa bozulsun sonuç değişmez.

10. Labirent Çalışma Sorusu

Hoca bir çalışma sorusu daha verdi: Bir labirentte A*, best-first search, UCS ve DFS hangi kareleri dener? Birinci ve ikinci durum olmak üzere iki labirent gösterildi. Soru şu tarzdadır: "Bu algoritmayı çalıştırdığımda üç iterasyon sonra BFS hangi durumda olur?"

Hoca BFS için bir örnek kural verdi: önce sağ, sonra yukarı, sonra aşağı, sonra sol. Bunun sezgisel bir kural olmadığını, tamamen algoritmik bir direktif olduğunu vurguladı. BFS'nin işleyişi şöyledir: Kuyrukta başlangıç B vardır. B çıkarılır; B1 (sağ), B2, B3 kuyruğa eklenir. Sonra B1 çıkarılır; B1'den gidilebilecekler (B4, B5, B6 gibi) kuyruğa eklenir. Kuyrukta B2, B3, B4, … vardır. Sonra B2 çıkarılır ve böyle devam edilir. Hocanın benzetmesiyle BFS, suya atılan bir taşın dalgaları gibi etrafa genişleyerek ilerler.

DFS için de aynı sıra (sağ, yukarı, aşağı, sol) verilirse DFS önce sağa gider, gidebildiği kadar bir yönde ilerler. Gidecek başka yer kalmayınca geri gelir ve başka bir koldan devam eder. UCS'de her adımın maliyeti eşit kabul edilebilir.

11. A*'ın Optimalliği

11.1 Koşul

$f(n) = g(n) + h(n)$ fonksiyonunda $h(n)$ gerçek maliyeti daha düşük tahmin ediyorsa (gerçek maliyeti aşmıyorsa) ve bütün maliyetler pozitifse A* optimal çözümü bulur. Gerçek maliyeti daha düşük tahmin etmek ne demektir? Harita probleminde bunun yolu kuş uçuşu mesafeyi kullanmaktır: İki şehir arasındaki gerçek yol, kuş uçuşu mesafeden kısa olamaz. Hocaya göre

“bunda tartışılacak bir şey yok”. Benzer problemlerde de yapılması gereken şey, gerçek maliyetin altında, iyimser ve pozitif bir tahmin veren bir $h(n)$ seçmektir. Hoca eşitliğin de kabul edilebileceğini belirtti.

11.2 İspat

Hoca ispatı şöyle kurdu:

- G optimal hedef olsun. Ağaçta bütün durumları çizdiğinizde en iyi çözüm budur. Örneğin maliyeti 20.
- G_2 alt-optimal (suboptimal) bir hedef olsun. Yine hedefe gider ama daha pahalıdır; örneğin maliyeti 21.
- A*'ın kuyruğunda, G'ye giden optimal yol üzerindeki bir n düğümü bulunsun. Aramaya devam ederken G_2'ye götüren yola değil de n'ye gelirseniz, n'den G'ye gidebilirsiniz.

Gösterilmesi gereken: A*, G_2'yi n'den önce açmaz. Bunun için $f(n) < f(G_2)$ olduğu gösterilir.

G_2 bir hedef olduğu için $h(G_2) = 0$'dır. Ayrıca G_2 alt-optimal olduğu için:

$$f(G_2) = g(G_2) + h(G_2) = g(G_2) > g(G)$$

h kabul edilebilir olduğu için $h(n)$, n'den G'ye gerçek maliyeti aşmaz. n, G'ye giden optimal yolun üzerinde olduğu için:

$$f(n) = g(n) + h(n) \leq g(n) + (\text{n'den G'ye gerçek maliyet}) = g(G)$$

İki eşitsizlik birleşince:

$$f(n) \leq g(G) < f(G_2)$$

Demek ki n her zaman G_2'den önce seçilir. n seçilince de arama G'ye doğru devam eder. Böylece A* gerçekten optimum çözüme ulaşır.

> **Hocanın vurgusu:** Optimallik önemli bir konudur. Her önüne gelen h fonksiyonu sizi optimum sonuca götüremez; h'yi doğru seçmek gerekir. Kitapta bununla ilgili güzel açıklamalar var, okumanız tavsiye edilir.
>
> **Ek bilgi (derste anlatılmadı):** Yukarıdaki ispat ağaç araması (tekrar eden durumların ayrı yollar olarak kuyruğa girdiği arama) içindir; bu derste yapılan izlemeler de böyledir. Bir kez açılan durumu kapalı listeye alıp bir daha açmayan graf aramasında, kabul edilebilirliğe ek olarak tutarlılık (consistency) koşulu da istenir: her n ve ardılı n' için $h(n) \leq c(n, n') + h(n')$. Kuş uçuşu mesafe bu koşulu da sağlar. Russell ve Norvig'in kitabında ayrıntılı anlatılır.

12. Uygulama: Algoritmaların Karşılaştırılması ve Robot Keşfi

12.1 Adım Sayısı ve Maliyet Karşılaştırması

Hoca, başta gösterdiği uygulamaya geri döndü. Algoritmaları adım sayısı ve maliyet açısından karşılaştırdığınızda, sezgisel aramalar kör aramalarla aynı (optimum) maliyeti bulabilir. Slayttaki örnekte UCS ile A* aynı yolu buluyor; ama UCS bunu çok fazla adımda yapıyor. Eklenen iyi bir sezgisel fonksiyon sayesinde A*'ın 7 adımda bulduğu şeyi UCS yaklaşık iki katı adımda buluyor. Best-first search de bu örnekte aynı yolu bulmuş.

Hoca uygulamayı yeniden çalıştırdı. 45 düğüm ve 50 yolla denediğinde graf çok seyrek kaldığı için yol bulunamadı. Düğüm sayısını 35'e indirince UCS yine optimum yolu buldu ama 32 adım attı. A* ve best-first de aynı sonuca daha az adımda ulaştı.

> **Hocanın vurgusu:** Best-first search her zaman yerel minimuma takılacak diye bir kaide yok. Öyle durumlar olur ki sizi daha kısa sürede doğru sonuca götürür.

12.2 Robot Keşfi

Hoca son olarak, slaytların sahibi olan Fatih Hoca'nın paylaştığı gerçek bir uygulamayı gösterdi: çok robotlu keşif. Bir ortamda arama yapmadan önce o ortamın haritasını çıkarmak gerekir. Bunun için bir algoritmayla bütün ortam gezilir; burada kör arama veya A* gibi algoritmalar kullanılabilir. Ardından hedefler belirlenir. Örneğin bir yerde yangın var ve robotun gidip yangını söndürmesini istiyorsunuz. Haritayı çıkarmadan o hedefe gitmek mümkün değildir. Gösterilen 2019 tarihli çalışmada sekiz robot, haritayı daha hızlı çıkarmak için A* algoritması kullanıyor. Bunu BFS veya DFS ile yapmak çok daha zor olurdu. Makalede kullanılan h ve f fonksiyonları ayrıntılı veriliyor; hoca makaleyi okumayı önerdi.

A'ın farklı değiştirilmiş (modifiye) türleri de vardır. Hocanın danışmanlık yaptığı bir öğrenci ekibi, dronlar üzerinde A'ın bir türevini kullanarak Teknofest'te en iyi algoritma ödülünü almış. Hoca kullanılan türevin adını tam hatırlamadığını söyledi.

12.3 Sezgisel Aramanın Özeti

Hoca konuyu şöyle özetledi: Problem hakkındaki bilgi, hedefe ulaşmak için kalan maliyetin tahminini sunar. İyi sezgi, iyimser bir sezgidir. Hem arama süresini hem bellek kullanımını büyük oranda azaltır. A*, yapay zeka için anahtar bir teknolojidir.

Örnek Sorular

Bu derste video karesi yoktur. Sorular yalnız derste sözlü sorulan sorulardan alınmıştır.

1. (derste sözlü, 00:35:42) Romanya haritasında Arad'dan Bükreş'e best-first search ($f(n) = h(n)$, h: kuş uçuşu uzaklık) ile bulunan yol optimum mu?

Çözüm (derste verildi): Best-first search sırasıyla Arad (366), Sibiu (253), Fagaras (176) ve Bükreş'i (0) çeker. Yol Arad → Sibiu → Fagaras → Bükreş, maliyeti $140 + 99 + 211 = 450$'dir. Optimum değildir, çünkü Arad → Sibiu → Rimnicu Vilcea → Pitesti → Bükreş yolu $140 + 80 + 97 + 101 = 418$'dir. Algoritma yerel minimuma takılmıştır.

2. (derste sözlü, 00:45:28 ve 00:47:18) A* algoritması tam mıdır, optimal midir?

Çözüm (derste verildi): Tamdır, çünkü karar $f = g + h$'ye göre verildiğinden döngülü yolların f değeri sürekli büyür ve seçilmez (örneğin Arad → Sibiu → Arad için $f = 646$). Optimaldir, ancak $h(n)$ iyimser seçilirse (gerçek kalan maliyeti aşmazsa) ve tüm maliyetler pozitifse. h yanlış tasarlanırsa A* optimal sonucu vermeyebilir.

3. (derste sözlü, 01:00:45, çalışma sorusu) Siyah (B) ve beyaz (W) taşların yer değiştirdiği esnetilmiş problem için bir $h(n)$ önerin. Bir taş yanındaki boşluğa kayabilir veya yanındaki bir taşın üzerinden atlayarak boşluğa geçebilir.

4. (derste sözlü, 01:21:00) Slayttaki S–G grafında (Bölüm 9.5) A* ile S'den G'ye gidin. Her adımda kuyruğun durumunu ve kuyruktan çekilen yolu yazın.

Çözüm (derste verildi, Python ile doğrulandı): Bölüm 9.5'teki tablo. Çekilme sırası S, S-D, S-A, S-A-B, S-A-B-E, S-A-B-C, S-A-B-E-F, S-A-B-E-F-G. Bulunan yol S–A–B–E–F–G, maliyet 15.

5. (derste sözlü, 01:24:17, çalışma sorusu) Verilen iki labirentte A*, best-first search, UCS ve DFS hangi kareleri dener? Örneğin üç iterasyon sonra BFS hangi durumda olur? (Komşu sırası: sağ, yukarı, aşağı, sol.)

Kısmi çözüm (derste verildi): BFS'de kuyruktan B çıkar, B1, B2, B3 eklenir; B1 çıkar, B1'in komşuları (B4, …) eklenir; sonra B2 çıkar. BFS taş atılan su gibi genişleyerek, DFS ise bir yönde gidebildiği kadar gidip geri dönerek arar.

Dersin Toparlaması

Bu derste kör arama algoritmalarının en büyük zayıflığı, yani hedefe yaklaşmışken bilgisizlik yüzünden uzaklaşabilmeleri, rastgele harita uygulamasıyla gösterildi ve sezgisel aramaya geçildi. Sezgisel aramada bir sonraki düğüm, hedefe kalan maliyetin tahmini olan $h(n)$ kullanılarak seçilir. Best-first search yalnız $h(n)$'e bakar; hızlıdır, ama Romanya ve ızgara örneklerinde görüldüğü gibi yerel minimuma takılabilir, tam ve optimal değildir. A* ise harcanan maliyet

$g(n)$ ile tahmini kalan maliyet $h(n)$'yi toplar. Döngüleri kendiliğinden eler ve $h(n)$ iyimser olduğu sürece optimal yolu bulur. 8 taş örneğinde gerçeğe daha yakın tahmin veren baskın sezgisel fonksiyonun (h_2) çok daha az düğüm açtırdığı görüldü. S–G grafı üzerinden sınavda beklenen A* tablo biçimi gösterildi.

Gelecek hafta yerel arama (local search) algoritmalarına geçilecek. Her problem için kolayca bir $h(n)$ yazılamayabilir. Bu derste ayrık bir dünyada, dallanma sayısı belli bir arama ağacıyla çalışıldı. Gerçek dünyadaki bazı problemlerde ise süreklilik ve sonsuzluk vardır; orada A* gibi algoritmalar bile yeterince verimli olmayabilir. Bu yüzden doğadan esinlenen yöntemler, genetik algoritma ve fonksiyonların minimum ve maksimum noktalarını bulma gibi konular işlenecek. Hoca konunun kitabın üçüncü bölümünden tekrar edilmesini önerdi.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.